earlier in this series? This extension is typically associated with SageMath worksheets, commonly used in platforms like CoCalc and JupyterLab. However, for standard SageMath scripts, the *.sage* extension is more appropriate.

A brief history of the beginnings of modern cryptography

Before delving into more complex codes, let me take you through a brief history to understand the evolution of cybersecurity and cryptography.

While the two World Wars brought immense human suffering, they also drove significant advancements in science and technology, resulting in inventions like nuclear energy, digital computers, airplanes, submarines, and more. Cryptography was one such field that underwent a revolutionary transformation during this period.

In World War I, substitution ciphers like the Playfair cipher were commonly used. A pivotal moment in the war occurred when British intelligence intercepted and deciphered the Zimmermann Telegram—a secret communication from the German ambassador to Mexico. This breakthrough played a significant role in the United States entering the war.

World War II saw the Germans rely heavily on the Enigma machine, an electromechanical device they believed to be unbreakable. However, much like the French during the Franco-Prussian War of 1870, the Germans underestimated their adversaries. British intelligence successfully cracked the Enigma's encryption scheme, marking a turning point in the Allies' victory in World War II.

This monumental achievement was led by Alan Turing, one of the most influential figures in computer science. His work not only contributed to the Allies' success but also laid the foundation for modern computing and cryptography. The Hollywood film 'The Imitation Game' vividly portrays the cracking of the Enigma codes and the tragic life of this extraordinary individual.

World War II also accelerated the development of digital computers, which paved the way for more sophisticated encryption schemes and heralded the beginning of modern cryptography.

Transposition ciphers

Let us now explore another classical encryption scheme, the Rail Fence cipher, before moving onto modern encryption techniques. The Rail Fence cipher is a transposition cipher that rearranges the positions of characters without altering the characters themselves. For example, the word 'listen' can be rearranged to form 'silent'.

To understand the Rail Fence cipher, let us walk through a simple example. In a simplified version of this cipher, the plaintext is written column by column into a grid and then read row by row to generate the ciphertext. The number of rows in the grid, known as the number of rails, is determined by the encryption scheme designer. The example below demonstrates how the plaintext 'CONSIDERABLE' is encrypted using three rails.

```
Rail 1: C S E B
Rail 2: O I R L
Rail 3: N D A E
```

Upon reading the plaintext row by row, we obtain the ciphertext 'CSEBOIRLNDAE'. Decryption follows a similar process: the ciphertext is written row by row, and then read column by column to reconstruct the original plaintext.

Now, let us consider the program *railfence.sage*, which implements this simplified version of the rail fence encryption scheme. To simplify further, we assume that only uppercase letters are used, and no whitespaces are present. The code below performs the encryption process in the program *railfence.sage* (line numbers are included for clarity).

```
1.   def rail_fence_encrypt(plaintext, num_rails):
2.       rails = [''] * num_rails
3.       for i, char in enumerate(plaintext):
4            rails[i % num_rails] += char
5.       ciphertext = ''.join(rails)
6.       return ciphertext
```

Now, let us go through the code line by line to understand how it works. Line 1 defines a function named *rail_fence_encrypt*. It takes two arguments: *plaintext*, which contains the text to be encrypted and *num_rails*, which specifies the number of rows (rails) to use in the Rail Fence cipher. Line 2 creates a list *rails* with *num_rails* empty strings, one for each rail. For example, if *num_rails = 3*, then rails will be [‘’, ‘’, ‘’]. These strings will store the characters assigned to each rail during encryption. Line 3 iterates over each character in the *plaintext*. The *enumerate( )* function provides the index of the character in the plaintext and the character itself. Line 4 distributes characters to rails. The result of '*i % num_rails*' cycles through 0, 1, 2, ..., (*num_rails-1*), ensuring characters are distributed cyclically across the rails. Line 5 joins all the strings in the list *rails* into a single string, *ciphertext*. Line 6 returns the encrypted text (*ciphertext*) as the output of the function.

The code below performs the decryption process in the program *railfence.sage* (line numbers are included for clarity):

```
7.   def rail_fence_decrypt(ciphertext, num_rails):
8.       rail_lengths = [len(ciphertext[i::num_rails]) for i
             in range(num_rails)]
9.       rails = [ ]
```